

CascadeDecays and decayamplitude compatibility audit

Kinematics, dynamics ownership, and amplitude-convention validation

CascadeDecays.jl compatibility study

Table of contents

1	Executive result	3
2	Scope and method	4
2.1	Numerical closure	4
3	Compatibility matrix	6
4	Part 1: kinematic variables	8
4.1	Four-vectors, masses, and angles	8
4.2	Child order is the largest kinematic hazard	8
4.3	Initial frame and event lifetime	9
4.4	Cross-topology alignment	9
5	Part 2: lineshapes and form factors	10
5.1	The required entity boundary	10
5.2	There is no decayamplitude lineshape convention to deserialize	11
5.3	Breit–Wigner sign and denominator	11
5.4	Form-factor normalization	12
5.5	Event masses versus nominal channel masses	12
6	Part 3: amplitude construction	13
6.1	Shared conventions, proven numerically	13
6.2	Different computational organization	13
6.3	Model generation, parity, and identical particles	14
6.4	Output layout and intensity	14
7	Proposed neutral serialization contract	15
7.1	Import and export rules	16
8	Validation ladder for adapters	17
9	Prioritized conclusions	18

1 Executive result

This audit compares `CascadeDecays.jl` at commit `a372505560c4a8ebc52b5c44b0e15ac907e33476` with [KaiHabermann/decayamplitude v0.1.9](#) and its pinned kinematic dependency [decayangle v1.1.2](#).

The central result is positive: for the default `decayamplitude` helicity convention, the two frameworks use the same local helicity angles, standard Wigner $D(\text{phi}, \text{theta}, 0)$ convention, complex conjugation, Jacob–Wick particle-2 phase, LS-to-helicity transformation, and net $\sqrt{2J+1}$ factor per internal line. A one-event computation-graph probe agrees at every checked layer to better than $6\text{e-}16$.

The incompatibilities that matter for serialization are instead structural:

1. `decayangle` sorts children by default, while `CascadeDecays` treats written child order as physical.
2. `decayamplitude.DecayChain` owns an event and arbitrary Python lineshape closures; a `CascadeDecays` model is event-independent and receives a separate `KinematicPoint`.
3. `decayamplitude` has no inspectable lineshape or form-factor vocabulary. `CascadeDecays` separates a `propagator(s)` from a vertex form factor `ff(s0,s1,s2)`.
4. A `decayamplitude.Resonance` can sum several LS or helicity couplings at one vertex. A normal `CascadeDecays` vertex stores one recoupling term; importing a sum requires chain expansion or a new linear-combination vertex type.
5. Cross-topology Wigner alignment is automatic in `ChainCombiner` but opt-in, by external particle, in `KinematicTask`.

Consequently, serialization should target a versioned, framework-neutral intermediate representation. It should not pickle Python callables, record `Resonance.id`, or serialize Julia concrete type names as the physics model.

Two upstream API issues follow directly from the audit:

1. A propagator must be a first-class internal-line entity. It may own a table of decay channels, each with its own `l` and `s`, but its numerical contract is only $X(\text{sigma} \mid \text{parameters})$. The active production or decay vertex must not inject LS/helicity labels or daughter masses into that evaluation.
2. A vertex form factor belongs to the vertex and is evaluated from the three masses of the lines it connects, $F(m0,m1,m2 \mid \text{parameters})$, with the mass-versus-mass-squared convention and normalization declared explicitly.

The complete proposed issue texts are archived with this report in [decouple-propagators.md](#) and [vertex-form-factors.md](#).

2 Scope and method

The audit follows one event from four-vectors to a full helicity amplitude:

```
RamboOnDiet four-vectors
-> ordered topology and line invariants
-> local helicity frames and (phi, cos(theta))
-> LS coefficient x particle-2 phase
-> conjugated Wigner D
-> vertex factors
-> internal-helicity sum
-> propagators x sqrt(2J+1)
-> external-helicity amplitude
```

The fixture is a massive three-body event generated with `RamboOnDiet`, seed `0xDECODE`, parent mass 2.0, and daughter masses (0.4, 0.3, 0.2). The model uses ordered topology ((1,2),3), external doubled spins (0,1,1), root doubled spin 2, internal doubled spin 1, root coupling (2L,2S)=(0,2), isobar coupling (0,1), and unit lineshapes.

The checked inputs and outputs are in `event.toml` and `reference.toml`. The executable probes are `verify_rambo_event.jl`, `probe_cascade.jl` and `probe_decayamplitude.py`.

From the repository root, the Julia side runs as

```
julia --project=docs scripts/decayamplitude_compat/verify_rambo_event.jl
julia --project=. scripts/decayamplitude_compat/probe_cascade.jl
```

The Python probe requires the pinned `decayamplitude` checkout or installation, including `decayangle==1.1.2`, then runs as

```
python scripts/decayamplitude_compat/probe_decayamplitude.py
```

2.1 Numerical closure

Layer	CascadeDecays max error	decayamplitude max error
line masses and local angles	0	5.38e-16
representative graph terms	0	3.33e-16
all 12 external amplitudes	0	2.24e-16
spinful cross-topology alignment angles	0	<2e-14
all 12 aligned alternate-chain amplitudes	0	<2e-15

Layer	CascadeDecays max error	decayamplitude max error
-------	-------------------------	--------------------------

For the component $(2h_0, 2h_1, 2h_2, 2h_3) = (0, 0, -1, 1)$, the two internal-helicity paths are:

2lambda_R	root vertex	isobar vertex	product before spin normalization
-1	-0.3992180457	-0.3779612865 + 0.3230365043i	0.1508889661 - 0.1289620019i
+1	-0.0853909753	-0.3822072381 - 0.3266654404i	0.0326370488 + 0.0278942806i

Multiplying their sum by $\sqrt{2}$ gives $0.2595449794 - 0.1429313423i$ in both frameworks.

3 Compatibility matrix

Concern	CascadeDecays	decayamplitude v0.1.9	Verdict
four-vector storage	<code>FourVector(px,py,pz; E)</code>	array [..., (px,py,pz,E)]	same
invariant stored/routed	squared mass <code>s</code>	daughter masses passed; parent <code>sqrt(s)</code> usually closure-captured	adapter needed
local angle	first ordered child	first ordered child	same after ordering
default local rotation	<code>conj(D(phi,theta,0))</code>	<code>conj(D(phi,theta,0))</code>	same
optional <code>minus_phi</code>	absent	<code>conj(D(phi,theta,-phi))</code>	unsupported variant
particle-2 helicity phase	exact $(-1)^{(j_2-\lambda_{\text{bda}2})}$	same expression	same
particle-2 frame transport	explicit particle-2 instruction	<code>R_y(-pi)</code> on daughter 2	same numerically
LS-to-helicity coefficient	<code>sqrt((2L+1)/(2J+1))</code> and two CGs	same	same
internal spin normalization	explicit product <code>sqrt(2J_R+1)</code>	recursive factors with root cancellation	same net result
child ordering	written order is physical	sorted by default	different, high risk
event ownership	separate model and kinematic point	momenta stored in chain	different
lineshape interface	typed callable of parent <code>s</code>	arbitrary callable of <code>(2L,2S,params,...)</code>	different
form factor	separate vertex callable	normally folded into lineshape callback	different
root dynamics	no root propagator	root is a Resonance and its lineshape is evaluated	different
multiple LS terms per vertex	one ordinary recoupling term	coherent dictionary sum	different
topology alignment	requested external spins only	automatic for all final axes in combiner	activation differs
amplitude container	dense array, finals then root	dict keyed by final helicities; root is input	adapter needed
unpolarized intensity	sum over all axes, no mother average	same, no mother average	same

Concern	CascadeDecays	decayamplitude v0.1.9	Verdict
event batching	one <code>KinematicPoint</code> per event	JAX/NumPy arrays may carry batch axes	performance/API difference
identical-particle handling	explicit chains and shared coefficients	<code>Particle.type_id</code> influences sorting/topology generation	different
stable model identity	value-like topology/payloads	process-global creation-order resonance IDs	different

4 Part 1: kinematic variables

4.1 Four-vectors, masses, and angles

Both frameworks use $(\mathbf{p}_x, \mathbf{p}_y, \mathbf{p}_z, E)$ and metric $(+, -, -, -)$. On an event already in the mother rest frame, CascadeDecays `CurrentFrame()` and decayangle's default `helicity` convention produce identical angles. For the fixture:

Vertex	phi	cos(theta)
root ((1,2),3)	-2.5149425199858495	-0.2091643181654772
isobar (1,2)	1.4144112448600463	0.011170731326575443

CascadeDecays stores one squared invariant per line. `decayamplitude` recomputes masses from sums of final four-vectors, and its recently added callback path passes daughter masses rather than squared masses. A neutral schema should use `s` internally and state every callable's expected transformation explicitly.

4.2 Child order is the largest kinematic hazard

CascadeDecays preserves bracket order. Thus (1,2) and (2,1) differ in local angles, particle-2 phases, and frame paths.

`decayangle` defaults to `config.sorting="value"`: composite daughters precede single particles and otherwise values are sorted. The constructed `topology.tuple`, not the user's input tuple, is the physics topology. In addition, `ResonanceDict.stable_value` sorts the flattened descendant set.

This does **not** make ordered amplitudes impossible in `decayamplitude`. `decayangle` sorting can be disabled before topology construction (or replaced by an identity ordering function). The resulting `Topology` retains ordered `node.daughters`, and `DecayChainNode.amplitude` uses that order for angles, helicities, the particle-2 phase, and recursion. Our numerical comparison uses identical ordered topologies and confirms this path.

The remaining limitation is narrower: `ResonanceDict` uses an unordered descendant set only to find a payload. A single dictionary therefore cannot hold two different payload entries keyed by (1,2) and (2,1). This matters only when the payload itself is orientation-specific. It can be worked around today by constructing per-chain resonance dictionaries, but a general adapter should address payloads by a topology-local ordered vertex ID rather than by `ResonanceDict.stable_value`.

For example, an adapter can preserve an input topology as follows:

```

from decayangle.config import config as decayangle_config
from decayangle.decay_topology import Topology

decayangle_config.sorting = "off" # set before constructing any Node/Topology
topology = Topology(0, decay_topology=((2, 1), 3))
assert topology.tuple == ((2, 1), 3)

```

An exporter must therefore:

1. serialize `Topology.tuple` after construction;
2. write ordered `children=[child1,child2]` explicitly at every vertex;
3. record the decayangle ordering policy for provenance; and
4. never recover child order from a sorted descendant-set key; and
5. use per-chain dictionaries or ordered vertex IDs for orientation-specific payloads.

4.3 Initial frame and event lifetime

`decayamplitude.DecayChain` stores momenta and caches `helicity_angles`. Lineshape factories in its examples also close over an event-dependent parent mass array. Replacing or mutating the momenta after construction can therefore leave cached or captured values inconsistent.

`CascadeDecays` keeps the model static. `DecayChainKinematics` or `KinematicPoint` is constructed per event. `HelicityRootFrame()` handles general moving-parent four-vectors, with a current-frame fallback when the mother is already at rest. `decayangle` expects the start node to be at rest for its recursive boost path; the caller normally invokes `topology.to_rest_frame(momenta)` first.

The serialized model should contain no event arrays. Event serialization, if needed, belongs in a separate object with a declared frame.

4.4 Cross-topology alignment

Both implementations calculate relative Wigner rotations by comparing the helicity-frame paths of a final particle. `ChainCombiner` uses the first chain as the reference and aligns later chains automatically. `CascadeDecays` requires `KinematicTask(...; reference_topology=..., wigner_finals=...)`; unrequested axes remain unrotated.

The model schema must name a reference topology and list the external particle IDs whose helicity axes are aligned. Importing a `ChainCombiner` should request all spinful final particles in `wigner_finals`.

The fixture also checks an alternate $((1,3),2)$ chain aligned to the $((1,2),3)$ reference on both spin-1/2 final particles. All 12 aligned complex amplitudes agree within $2\text{e-}15$; the Euler-angle representatives agree up to their expected floating-point and periodic ambiguities.

5 Part 2: lineshapes and form factors

5.1 The required entity boundary

The current `decayamplitude` API evaluates a callable called `lineshape` from inside the vertex LS/helicity sum. That placement conflates three independently serializable objects:

```
\mathcal{A}_c =
\left[\prod_{R \in c} X_R(\sigma_R \mid p_R)\right]
\sum_{\{\lambda_R\}}
\left[\prod_{v \in c}
H_v, F_v(m_{v0}, m_{v1}, m_{v2} \mid p_v), D_v^*
\right].
```

The ownership and evaluation rules should be:

Entity	Owns	Event-dependent numerical inputs	Must not know
propagator/internal line	analytic model, pole parameters, independently declared decay channels and their $\mathbf{l}, \mathbf{s}$	its own actual σ	the active vertex term, helicities, connected event masses
decay channel	channel particles, nominal threshold masses, channel $\mathbf{l}, \mathbf{s}$, channel parameters	none unless the analytic model explicitly defines otherwise	which vertex term happened to request the propagator
vertex	ordered parent/child line IDs, helicity or LS couplings, form factor and its normalization	the connected (m_0, m_1, m_2) and angles	an internal-line denominator

Thus “the propagator knows its channels” does **not** mean `propagator(l, s, d1_mass, d2_mass, ...)`. It means that channel records are part of the propagator’s declarative configuration. Evaluation is still `X( $\sigma$  | parameters)`, and the channel sum or running-width policy is internal to that model. Conversely, `F(m0, m1, m2 | parameters)` is a vertex factor because its breakup momentum is defined by exactly those three connected lines.

This separation also resolves the particle-order conclusion. Ordered child identity is a property of the vertex. A nonsymmetric vertex form factor receives `m1` and `m2` in that stored order. A propagator does not use vertex child order at call time; any ordered or unordered channel-particle convention is

declared inside its own channel record. Therefore particle order can be made exact in `decayamplitude`, but only after disabling automatic topology sorting and representing these entities separately. The present arbitrary callback API does not provide enough structure to repair or serialize the distinction reliably.

5.2 There is no `decayamplitude` lineshape convention to deserialize

`decayamplitude.Resonance` accepts an arbitrary callable. For an LS model the call is effectively

```
lineshape(two_L, two_S, *parameters, d1_mass=?, d2_mass=?)
```

The `d1_mass` and `d2_mass` keywords are passed only when signature inspection finds both names or `**kwargs`. The parent resonance mass is not a normal argument. Examples capture it in a closure. This is executable Python state, not a portable model description.

`CascadeDecays` separates two roles:

- an internal-line propagator receives the parent line’s event-dependent `s`;
- a vertex form factor receives `(s0,s1,s2)` for its ordered parent and children.

For interchange, the schema should expose a named `mass_argument` convention. The conceptual vertex interface above uses masses `(m0,m1,m2)`; the current `CascadeDecays` callable receives squared masses `(s0,s1,s2)`. Conversion between them is mechanical only when this convention is explicit.

Built-in `HadronicLineshapes.BreitWigner` stores pole parameters, nominal daughter masses, ordinary orbital `L`, and radius. This distinction exposes an important units trap: `decayamplitude` passes doubled `2L` to the callback, while `HadronicLineshapes.BreitWigner.l` is ordinary integer `L`.

5.3 Breit–Wigner sign and denominator

`decayamplitude` does not define a Breit–Wigner. Its examples use

```
X_{example}(s) = \frac{1}{s-m_0^2+i\sqrt{s}\,\Gamma}.
```

`CascadeDecays`’ standard dependency starts from

```
X_{Cascade}(s) = \frac{1}{m_0^2-s-i\,m_0\Gamma(s)}.
```

With identical imaginary terms these differ by a global minus sign, absorbable into one chain coefficient. The shipped forms are not otherwise identical: `sqrt(s)` versus `m0`, and constant versus running width. At the fixture’s `s=1.9919876737`, `m0=1.5`, and `Gamma=0.2`, the `decayamplitude` example gives `-1.76419-1.93009i`, while `CascadeDecays`’ P-wave running-width object with daughter masses `(0.4,0.3)` and radius `1.5` gives `1.84333+1.93558i`.

The recording test added with daughter-mass support even sets its local BW mass to `d1_mass`. That is suitable for testing argument plumbing, but physically it uses the first child’s four-vector mass in the resonance denominator and must not become a serialization recipe.

Every portable lineshape must declare its analytic `type`, sign convention, parent invariant source, width formula, pole parameters, channel masses, and analytic-continuation policy. Unknown callables must be rejected or exported as explicitly nonportable extensions.

The root is another `Resonance` in `decayamplitude`, so its lineshape is evaluated at the mother vertex. `CascadeDecays` has no propagator on the root line. A constant root lineshape maps directly. A nonconstant factor can map to the root vertex form factor when it is declarative and factorizes from the recoupling; an LS-dependent or otherwise opaque production factor requires chain expansion or a separate top-level dynamics field.

5.4 Form-factor normalization

`HadronicLineshapes.BlattWeisskopf{L}(d)` returns the unnormalized threshold factor $F_L(q)$. A `CascadeDecays` vertex multiplies exactly the callable value; it does not divide by $F_L(q_0)$. The built-in running width separately normalizes its channel coupling at the pole, which introduces the appropriate $F_L(q)^2/F_L(q_0)^2$ behavior inside the width.

At the fixture isobar mass, for $L=1$, $d=1.5$, and nominal daughter masses $(0.4, 0.3)$:

```
F_1(q)      = 0.6758008537
F_1(q0)     = 0.7045380700
F_1(q)/F_1(q0) = 0.9592112655
```

Thus $F_L(q)$ and $F_L(q)/F_L(q_0)$ are measurably different models. The neutral schema needs a required normalization enum such as `raw`, `pole`, or `explicit_reference`, plus the definition of q_0 . Because `decayamplitude` folds this into arbitrary code, it cannot be inferred reliably.

5.5 Event masses versus nominal channel masses

Both frameworks may compute kinematic vertex masses from the event. They should not silently use reconstructed external four-vector masses as the nominal threshold masses or in a pole-normalization denominator. The recommended default is:

- parent `s`: event invariant of the addressed internal line;
- stable daughter masses in width and `q0`: serialized nominal particle masses;
- event daughter invariants: available only to explicitly dynamic vertex factors;
- any other choice: opt-in `mass_policy` recorded in the model.

`decayamplitude` applies `nan_to_num` to daughter masses before calling a lineshape, silently replacing NaN and infinite values by zero. `CascadeDecays` does not apply an equivalent model-level replacement; the selected lineshape controls threshold and complex-continuation behavior. A serializer should store the numerical-domain policy and adapters should reject invalid event masses by default rather than silently changing them.

6 Part 3: amplitude construction

6.1 Shared conventions, proven numerically

Both default amplitudes use

```
V^{\lambda_0\lambda_1\lambda_2} =  
D^{J_0*}_{\lambda_0,\lambda_1-\lambda_2}(\phi,\theta,0)  
(-1)^{J_2-\lambda_2}  
H_{\lambda_1\lambda_2}.
```

Their Wigner implementation is the standard $D=\exp(-im\phi) d(\theta) \exp(-im'\psi)$, followed by complex conjugation in the amplitude. `decayamplitude`'s optional `minus_phi` mode instead uses $D(\phi,\theta,-\phi)$ and must be marked as a separate convention.

The LS transformation is also identical:

```
H_{\lambda_1\lambda_2}^{\{LS\}} =  
\sqrt{\frac{2L+1}{2J+1}}  
\langle J_1\lambda_1, J_2{-\lambda_2} | S, \lambda_1{-\lambda_2} \rangle  
\langle L0, S(\lambda_1{-\lambda_2}) | J, \lambda_1{-\lambda_2} \rangle.
```

`decayamplitude` multiplies $\sqrt{2J+1}$ at every recursive node and divides the root factor in `chain_function`; the net factors are exactly the internal-line factors that `CascadeDecays` multiplies explicitly.

6.2 Different computational organization

`decayamplitude` recursively generates internal helicities and returns a sum of products. `CascadeDecays` builds dense vertex tensors, multiplies them into a line-indexed tensor, and contracts internal axes. These are equivalent evaluation strategies, not convention differences.

The model spaces are different:

- `decayamplitude` coherently sums an arbitrary dictionary of LS terms, each with its own complex coupling and LS-dependent lineshape, at one **Resonance**;
- its direct-helicity scheme stores arbitrary (λ_1, λ_2) couplings;
- a normal `CascadeDecays` `Vertex(RecouplingLS(...), ff)` stores one LS term;
- `CascadeDecay` then supplies one complex coefficient per complete chain.

Importing a multi-term decayamplitude vertex into current CascadeDecays can be done by expanding the Cartesian product of vertex terms into separate chains. The coefficient of each expanded chain is the product of the selected vertex couplings. This is exact when every LS-dependent dynamic factor is represented on the corresponding expanded vertex or line, but it may grow combinatorially. A better long-term representation is a `LinearCombinationRecoupling` or an explicit list of vertex terms in the neutral schema.

Exporting CascadeDecays chain coefficients into decayamplitude has a coupling gauge freedom: a complete-chain coefficient can be placed at any one vertex. The adapter should use a declared rule, for example `coefficient_anchor=root`, and preserve a stable parameter ID so round trips do not move or duplicate fit parameters.

6.3 Model generation, parity, and identical particles

`MultiChain` forms the Cartesian product of candidate resonances at topology nodes and filters combinations whose generated couplings are invalid. `Resonance.generate_couplings` may materialize several parity-allowed LS states. CascadeDecays normally receives explicit chains; `minimal_ls_decay_chain` selects one allowed LS assignment. A portable file must store the selected LS terms, not a request to “regenerate allowed terms”, because generation order and selection policy are framework behavior rather than physics data.

decayamplitude’s `Particle.type_id` and `DecaySetup` use particle identity to sort and reduce generated topologies. CascadeDecays examples represent identical-particle permutations as explicit chains with tied coefficients. Serialization should therefore store explicit ordered chains plus parameter sharing. It may retain identical-particle groups as metadata, but should not ask an importer to regenerate or deduplicate the topology list.

`Resonance.id` is allocated from process-global class state and changes with construction and copying. Resonance names also participate in global parameter sharing. Neither mechanism is a stable serialization key. Parameter identity, display name, and sharing group must be separate explicit fields.

6.4 Output layout and intensity

CascadeDecays returns a dense array whose axes are final particles in label order followed by the root helicity. decayamplitude returns a dictionary keyed by a tuple of sorted final-particle helicities, with root helicity passed separately. Both unpolarized helpers sum absolute squares over initial and final helicities and do not divide by $2J_0+1$.

7 Proposed neutral serialization contract

The portable object should be a declarative model, not an executable object graph. A minimal versioned contract needs the following sections.

Section	Required content
schema	name, semantic version, feature flags
conventions	metric, four-vector order, doubled-spin units, ordered-child rule, Wigner-D definition, conjugation, particle-2 phase, spin normalization, intensity averaging
particles	stable ID, name, 2J, parity, nominal mass, final-state position or resonance identity
topologies	stable topology ID and ordered binary vertices; reference topology ID
alignment	external particle IDs to align and reference topology
parameters	stable parameter ID, value/default, real/complex, sharing group, optional bounds
chains	topology ID, complete-chain coefficient
vertices	parameter, vertex and internal-line assignments parent/child line IDs, coupling basis, explicit LS or helicity terms, form factor
propagators	analytic type, parent s source, pole parameters, channels, width formula, sign
form_factors	analytic type, ordinary L, radius units, momentum source, normalization and reference point
extensions	namespaced, explicitly nonportable framework-specific data

A representative convention block is:

```
{
  "angular_momentum_units": "twice_integer",
  "children": "ordered",
  "metric": "+---",
  "four_vector_order": ["px", "py", "pz", "E"],
  "helicity_frame": "jacob_wick_particle_2",
  "wigner_D": "exp(-i*m*phi)*d(theta)*exp(-i*mp*psi)",
  "local_euler": ["phi", "theta", 0],
```

```

"amplitude_uses_conjugate_D": true,
"particle_2_phase": "(-1)^(j2-lambda2)",
"internal_spin_normalization": "product_sqrt_2J_plus_1",
"initial_spin_average": false
}

```

7.1 Import and export rules

For decayamplitude to neutral form:

1. materialize the post-sorting `Topology.tuple`;
2. replace process-global resonance IDs with semantic IDs;
3. separate event arrays from the model;
4. materialize every generated LS/helicity term and parameter reference;
5. recognize only registered lineshape/form-factor factories;
6. reject opaque closures unless a namespaced extension is explicitly allowed;
7. record `helicity` versus `minus_phi` and the reference topology.

For neutral form to CascadeDecays:

1. validate the convention block before construction;
2. preserve child order exactly;
3. map doubled LS 2L to ordinary L only in dynamics objects;
4. construct event-independent propagators from parent `s`;
5. expand multi-term vertices or construct a linear-combination recoupling;
6. request Wigner alignment for every declared spinful external particle;
7. permute external axes to finals-then-root.

For neutral form to decayamplitude:

1. set `decayangle_config.sorting="off"` before constructing any topology, then verify that every constructed `Topology.tuple` equals the serialized ordered tree;
2. construct registered, named dynamics factories rather than closures;
3. place complete-chain coefficients according to the declared anchor rule;
4. convert parent `s` to `sqrt(s)` only at a callback boundary that requires it;
5. expose nominal daughter masses separately from event daughter invariants;
6. construct per-chain resonance dictionaries when two chains require different payloads for opposite orderings of the same descendant set.

8 Validation ladder for adapters

Every supported model should pass these tests in order:

1. topology and ordered-child identity;
2. all line $\mathbf{s}$ values on one RamboOnDiet event;
3. every local $(\mathbf{phi}, \cos(\mathbf{theta}))$;
4. each Wigner matrix element on used helicities;
5. raw LS/helicity coefficient before particle-2 phase;
6. coefficient after particle-2 phase and form factor;
7. each vertex factor;
8. every internal-helicity path before summation;
9. propagator and spin-normalization products;
10. all external-helicity amplitudes;
11. aligned amplitudes across at least two topologies;
12. a small event sample comparing complex amplitudes up to only explicitly declared global phase or normalization freedoms.

The checked fixture and probes implement levels 1–10 for one nontrivial chain. They also implement level 11 with a second topology and two spinful external particles. The next implementation milestone should add registered Breit–Wigner and both raw and pole-normalized Blatt–Weisskopf cases.

9 Prioritized conclusions

1. Do not change CascadeDecays' default `D`, particle-2 phase, LS formula, or `sqrt(2J+1)` normalization to match `decayamplitude`; they already match.
2. Make ordered child identity and post-sorting topology export the first adapter invariant.
3. Introduce a registered dynamics vocabulary before attempting general model serialization. Arbitrary callables cannot round-trip safely.
4. Separate nominal particle masses, event invariants, and pole-reference masses in the schema.
5. Make form-factor normalization mandatory, never inferred.
6. Add a linear-combination vertex representation or accept exact but potentially large chain expansion.
7. Use semantic parameter IDs and explicit sharing; never serialize `decayamplitude.Resonance.id`.
8. Treat `minus_phi`, nonconstant root lineshapes, and opaque callables as unsupported features until their mappings are specified and tested.
9. Refactor `decayamplitude` so propagators are internal-line entities evaluated only as `X(sigma | parameters)`, while their channel `l,s` metadata remains declarative and independent of vertex LS terms.
10. Make vertex form factors explicit objects of the connected `(m0,m1,m2)`, with ordered children and normalization recorded in the model.

This is a source and numerical audit of the pinned versions above. Arbitrary user-supplied Python and Julia callables can implement additional conventions; those cannot be classified without registering and inspecting their semantics.